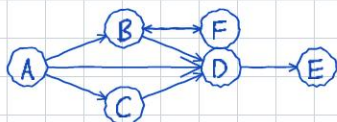


Depth-First Search (DFS)

DFS explores as far as possible along each branch before backtracking. It uses a stack (implicit via recursion or explicit) to track the frontier.



Recursive Algorithm

DFS(G):

time $\leftarrow 0$

for each vertex u in G :

color[u] $\leftarrow$ WHITE

for each vertex v in G :

if color[u] == WHITE:

DFS-VISIT(u)

DFS-VISIT(u):

time $\leftarrow$ time + 1

d[u] $\leftarrow$ time // discovery

color[u] $\leftarrow$ GRAY

for each v in Adj[u]:

if color[v] == WHITE:

parent[v] $\leftarrow$ u

DFS-VISIT(v)

color[u] $\leftarrow$ BLACK

time $\leftarrow$ time + 1

f[u] $\leftarrow$ time // finish

Discovery / Finish Times (example above, starting from A)

Vertex	d	f
---	---	---
A	1	12
B	2	9
C	10	11
D	3	6
E	4	5
F	7	8

Edge classification

For each edge ($u \rightarrow v$), classify by $\text{color}[v]$ when first explored:

- Tree edge - v is WHITE (discovered via this edge). Forms the DFS forest.
- Back edge - v is GRAY (ancestor still on stack). Indicates a cycle.
- Forward edge - v is BLACK and v is a descendant of u .
- Cross edge - v is BLACK and v is NOT a descendant of u .

In our example:

- Tree: $A \rightarrow B$, $B \rightarrow D$, $D \rightarrow E$, $B \rightarrow F$, $A \rightarrow C$
- Back: $F \rightarrow B$ (B is gray, on the recursion stack)
- Forward: $A \rightarrow D$ (D is black, descendant of A)
- Cross: $C \rightarrow D$ (D is black, not a descendant of C)

Key rule: $u.d < v.d < v.f < u.f$ iff v is a descendant of u in the DFS tree.

Applications

- Cycle detection: a graph has a cycle iff DFS finds a back edge.
- Topological sort: process vertices by decreasing finish time (reverse of finish order). Valid only for DAGs.
- Strongly Connected Components (SCCs): run DFS, transpose graph, process vertices in decreasing finish order (Kosaraju's or Tarjan's algorithm).
- Path finding, maze solving, connected components in undirected graphs.

Time complexity

Each vertex is discovered and finished once $\rightarrow O(V)$.

Each edge is examined once (directed) or twice (undirected) $\rightarrow O(E)$.

Total: $O(V + E)$

This is optimal - we must examine every vertex and edge at least once.

DFS goes deep first - use recursion or explicit stack